

第 6 章：DQN (Deep Q-Network, 深度 Q 网络) 与经典改进

本章符号说明

符号/缩写	English	中文	含义
DQN	Deep Q-Network	深度 Q 网络	用神经网络近似动作价值函数 $Q(s,a)$ 。
$Q(s,a)$	Action-value Function	动作价值函数	状态 s 下动作 a 的长期价值； Q 表示 action quality。
y	TD Target / Bellman Target	TD 目标 / Bellman 目标	用于监督当前 Q 网络的目标值。
$L(\theta)$	Loss Function	损失函数	网络参数 θ 的训练损失。
θ	Online Network Parameters	在线网络参数	当前正在训练的 Q 网络参数。
θ^-	Target Network Parameters	目标网络参数	滞后更新的 target network 参数。
p_i	Priority	优先级	Prioritized Replay 中第 i 条样本的采样优先级。
$P(i)$	Sampling Probability	采样概率	replay buffer 中样本 i 被采样的概率。
N	Buffer Size / Sample Count	buffer 大小 / 样本数	importance sampling weight 中的数据规模。
w_i	Importance Sampling Weight	重要性采样权重	修正非均匀采样 bias 的权重。
α / β	Priority Exponent / IS Correction Exponent	优先级指数 / 重要性采样修正指数	Prioritized Replay 中控制采样偏斜和 bias 修正的参数。
TD	Temporal Difference	时序差分	DQN 使用 TD/Bellman target 训练 Q 网络。
RL	Reinforcement Learning	强化学习	DQN 是 deep RL 的经典 value-based 算法。
SGD	Stochastic Gradient Descent	随机梯度下降	用 mini-batch 优化神经网络的训练方法。

符号/缩写	English	中文	含义
IS	Importance Sampling	重要性采样	Prioritized Replay 中用于修正非均匀采样 bias。
DDPG / TD3 / SAC / PPO	Deep Deterministic Policy Gradient / Twin Delayed DDPG / Soft Actor-Critic / Proximal Policy Optimization	深度确定性策略梯度 / 双延迟 DDPG / 软 Actor-Critic / 近端策略优化	连续控制或策略优化章节会出现的算法。

本章目标

本章学习 value-based deep RL 的代表算法 DQN。你需要掌握：

- DQN 的基本结构。
- experience replay。
- target network。
- Double DQN。
- Dueling DQN。
- Prioritized Experience Replay。
- DQN 的适用场景和局限。

本章主线

本章把第 3 章的 Q-learning 和第 5 章的神经网络函数逼近接起来。直接用网络拟合 Q-learning target 会因为样本相关、target 移动、过估计等问题不稳定。DQN 的 replay buffer、target network、Double DQN、Dueling DQN 和 Prioritized Replay 都是在修正这些具体不稳定来源。

DQN 解决什么问题

Q-learning 的表格更新是：

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

当状态空间巨大时，无法维护 Q-table。DQN 用神经网络近似 Q 函数：

$$Q(s, a; \theta)$$

如果动作空间是离散的，网络通常输入 s ，输出每个 action 的 Q 值：

$$\text{network}(s) \rightarrow [Q(s, a_1), Q(s, a_2), \dots, Q(s, a_n)]$$

DQN Loss

DQN 的 target：

$$y = r + \gamma \max_{a'} Q(s', a'; \theta)$$

如果 s' 是终止状态：

$$y = r$$

损失函数：

$$L(\theta) = \mathbb{E}[(y - Q(s, a; \theta))^2]$$

其中：

- θ 是 online network 参数。
- θ' 是 target network 参数。

实际实现中常用 Huber loss：

$$L = \text{Huber}(y - Q(s, a; \theta))$$

Huber loss 对异常 TD error 更稳。

从 Q-learning 到 DQN Loss 的推理过程

Tabular Q-learning 的更新是：

$$Q(s, a) \leftarrow Q(s, a) + \alpha [y - Q(s, a)]$$

其中：

$$y = r + \gamma \max_{a'} Q(s', a')$$

如果用神经网络 $Q(s, a; \theta)$ 代替表格，就不能直接“修改表格里的一個格子”。我们能做的是让网络当前输出靠近 target y 。

于是 RL 更新变成一个监督学习式回归问题：

```
input:  (s, a)
label:  y = r + gamma max Q_target(s', a')
predict: Q(s, a; theta)
loss:   (y - Q(s, a; theta))^2
```

这一步很关键：DQN 不是突然换了目标，它仍然在逼近 Bellman optimality backup，只是把 tabular update 改成了 neural network regression。

为什么 target 用 θ' 而不是当前 θ ？

- 如果 label 和 prediction 都由同一个快速变化的网络产生，训练目标会一直漂。
- target network 让 y 在一段时间内相对稳定。

- online network 只负责追这个相对固定的 Bellman target。

面试表达：

DQN 的 loss 可以看成把 Q-learning 的 TD target 当成监督信号。Replay buffer 解决样本相关性，target network 解决 bootstrap target 随参数同步漂移的问题。

DQN 训练流程

```
initialize online network Q(theta)
initialize target network Q(theta-) = Q(theta)
initialize replay buffer

for each step:
    choose action using epsilon-greedy
    execute action, observe r, s', done
    store (s, a, r, s', done) in replay buffer

    sample minibatch from replay buffer
    y = r + gamma (1 - done) max_a' Q(s', a'; theta-)
    minimize [y - Q(s, a; theta)]^2

    periodically update target network
```

Experience Replay

Replay buffer 存储 transition：

```
(s, a, r, s', done)
```

作用：

- 打破连续样本之间的相关性。
- 复用历史样本，提高 sample efficiency。
- 平滑训练数据分布。

面试表达：

DQN 中连续交互得到的样本高度相关，不适合直接做 SGD。Replay buffer 通过随机采样历史 transition，降低样本相关性，同时让每条经验被多次利用。

Target Network

如果 target 使用 online network：

$$y = r + \gamma \max_{a'} Q(s', a'; \theta)$$

那么 θ 每次更新都会改变 target, 训练会追逐一个移动目标。

DQN 使用 target network:

$$y = r + \gamma \max_{a'} Q(s', a'; \theta)$$

θ 每隔固定步数从 θ 同步一次, 或使用软更新。

面试表达:

Target network 让 bootstrap target 在一段时间内保持稳定, 降低训练目标和预测网络同步变化带来的震荡。

Double DQN

DQN 中的 target:

$$\max_{a'} Q(s', a'; \theta)$$

同一个 max 操作既选择动作又评估动作, 容易产生 overestimation bias。

Double DQN 将动作选择和动作评估拆开:

$$\begin{aligned} a^* &= \operatorname{argmax}_{a'} Q(s', a'; \theta) \\ y &= r + \gamma Q(s', a^*; \theta) \end{aligned}$$

其中:

- online network 负责选择动作。
- target network 负责评估该动作。

这样可以缓解 Q 值过估计。

Double DQN 为什么能缓解过估计

DQN 的问题来自 max operator。假设每个动作的 Q 估计都有噪声:

$$\text{estimated } Q = \text{true } Q + \text{noise}$$

当我们取 $\max_a Q(s', a)$ 时, 更容易选中“被正噪声高估”的动作。即使每个动作估计平均无偏, 最大值也会偏高。

普通 DQN 同一个 target network 同时做两件事:

1. 选择哪个动作最大。
2. 评估这个动作的价值。

Double DQN 把这两步拆开：

$$a^* = \operatorname{argmax}_a Q(s', a'; \theta)$$

$$y = r + \gamma Q(s', a^*; \theta')$$

这样选择动作时的噪声和评估动作时的噪声不完全相同，正噪声被重复放大的程度下降，因此 overestimation bias 更小。

面试表达：

Double DQN 的核心是 decouple action selection and action evaluation。DQN 的 max operator 容易选择被噪声高估的动作，Double DQN 用 online network 选动作，用 target network 估值，从而降低 overestimation bias。

Dueling DQN

Dueling DQN 将 Q 函数分解为状态价值和动作优势：

$$Q(s, a) = V(s) + A(s, a)$$

实际为了可辨识性，常用：

$$Q(s, a) = V(s) + A(s, a) - 1 / |\mathcal{A}| \sum_{a'} A(s, a')$$

直觉：

有些状态下，知道状态本身好不好比区分每个动作更重要。Dueling 结构让网络分别学习：

- $V(s)$ ：这个状态整体价值如何。
- $A(s, a)$ ：某个动作相对其他动作好多少。

适合动作之间差异不总是明显的场景。

Prioritized Experience Replay

普通 replay buffer 均匀采样 transition。

Prioritized Replay 更倾向采样 TD error 大的样本。

优先级：

$$p_i = |\delta_i| + \epsilon$$

采样概率：

$$P(i) = p_i^\alpha / \sum_k p_k^\alpha$$

为了修正非均匀采样带来的 bias，使用 importance sampling weight：

$$w_i = (1 / (N P(i)))^\beta$$

直觉：

TD error 大的样本说明当前模型学得不好，更值得训练。

Rainbow DQN

Rainbow DQN 组合了多个 DQN 改进：

- Double DQN。
- Dueling Network。
- Prioritized Replay。
- Multi-step Learning。
- Distributional RL。
- NoisyNet。

面试中通常了解组件即可，不一定要求完整推导。

DQN 的适用场景

适合：

- 离散动作空间。
- 可以通过 Q 值枚举动作。
- 游戏、简单控制、离散决策。

不适合：

- 高维连续动作空间。
- 需要直接输出连续控制量的任务。
- 极端稀疏奖励且探索困难的任务。

连续控制通常用 DDPG、TD3、SAC、PPO 等算法。

本章例子：从 CartPole 到 DQN

例子 1：CartPole

CartPole 的状态是：

状态维度	含义
cart position	小车位置
cart velocity	小车速度
pole angle	杆角度
pole angular velocity	杆角速度

动作只有两个：

向左推 / 向右推

这类任务可以用 DQN 学：

输入状态 s
输出 $Q(s, \text{left}), Q(s, \text{right})$
选择 Q 最大的动作

例子 2：为什么需要 replay buffer

如果 agent 连续玩游戏，连续样本高度相关：

第 t 帧：球在左边
第 $t+1$ 帧：球稍微往右
第 $t+2$ 帧：球继续往右

直接按时间顺序训练，网络容易被最近局部轨迹带偏。Replay buffer 随机采样历史 transition，让 mini-batch 更接近独立同分布。

例子 3：Double DQN 过估计小例子

假设真实下一状态两个动作价值都接近 10，但网络估计有噪声：

$Q(s', a1) = 10.1$
 $Q(s', a2) = 9.8$

普通 DQN 用同一个网络选动作和估值，会倾向选被噪声高估的动作。Double DQN 把“选动作”和“估值”拆开，可以降低这种系统性过估计。

面试表达：

DQN 的关键不是“神经网络版 Q-learning”这么简单，而是 replay buffer 和 target network 解决了深度网络训练时的相关样本和移动目标问题。

本章面试高频问题

1. DQN 相比 Q-learning 改了什么？

DQN 用神经网络近似 Q 函数，使 Q-learning 可以处理高维状态输入，例如图像。为了稳定训练，引入了 experience replay 和 target network。

2. DQN 为什么需要 replay buffer？

为了打破连续样本相关性，提高样本利用率，并使 mini-batch 训练更接近监督学习中的随机采样。

3. DQN 为什么需要 target network？

为了稳定 bootstrap target。如果 target 和 online prediction 使用同一网络，模型每次更新都会改变训练目标，容易震荡或发散。

4. Double DQN 解决什么问题？

解决 DQN 中 max operator 带来的 overestimation bias。它用 online network 选择动作，用 target network 评估动作，从而降低过估计。

5. Dueling DQN 的核心思想是什么？

将 Q 分解为状态价值 $V(s)$ 和动作优势 $A(s,a)$ ，让网络更好地学习状态本身价值和动作相对价值。

本章练习

1. 手写 DQN target 和 loss。
2. 解释 Double DQN 的 target 和 DQN target 的区别。
3. 画出 Dueling DQN 的网络结构。
4. 阅读一个 CleanRL DQN 实现，标出 replay buffer、target network、epsilon schedule 的位置。

[章节索引](#)

[← 上一章](#) [下一章 →](#)
